

Elastic Monte Carlo Tree Search with State Abstraction for Strategy Game Playing

Linjie Xu, Jorge Hurtado-Grueso, Dominic Jeurissen and Diego Perez Liebana

Department of EECS
Queen Mary University of London
London, UK
{linjie.xu,diego.perez}@qmul.ac.uk

Alexander Dockhorn

Faculty of EECS
Leibniz University Hannover
Hannover, Germany
dockhorn@tnt.uni-hannover.de

Abstract—Strategy video games challenge AI agents with their combinatorial search space caused by complex game elements. State abstraction is a popular technique that reduces the state space complexity. However, current state abstraction methods for games depend on domain knowledge, making their application to new games expensive. State abstraction methods that require no domain knowledge are studied extensively in the planning domain. However, no evidence shows they scale well with the complexity of strategy games. In this paper, we propose Elastic MCTS, an algorithm that uses state abstraction to play strategy games. In Elastic MCTS, the nodes of the tree are clustered dynamically, first grouped together progressively by state abstraction, and then separated when an iteration threshold is reached. The elastic changes benefit from efficient searching brought by state abstraction but avoid the negative influence of using state abstraction for the whole search. To evaluate our method, we make use of the general strategy games platform Stratega to generate scenarios of varying complexity. Results show that Elastic MCTS outperforms MCTS baselines with a large margin, while reducing the tree size by a factor of 10. Code can be found at <https://github.com/egg-west/Stratega>

Index Terms—State Abstraction, Monte Carlo Tree Search, Game Artificial Intelligence, Strategy Games

I. INTRODUCTION

The design of artificial intelligence (AI) for strategy game playing is a challenging problem. Human players can handle strategy games without much training when they possess knowledge about the game rules, opponent behavior, etc. AI solutions have different ways to gain knowledge about the game. Given domain knowledge, heuristics or scripts can be created. Heuristics can be used to guide the search of search-based AI agents [1]. In combination with scripts, agents are able to deal with more complex strategy games [2]. However, those scripts are rigid and limited by the programmer’s ability to capture the complexity of decision-making in strategy games (as most rule-based systems are). These scripts, once implemented, are static and they cannot adapt to new features introduced in the game, which is something typical during the game development cycle. In this case, these scripts need to be repeatedly updated to introduce new design updates.

An approach to get knowledge about the game is using a forward model. A forward model is a simulator that returns the next state given a state and an action. By simulating future states with the forward model, agents can sample different

playing trajectories and store this knowledge in a data structure such as a tree [3]. However, the size of the search space in strategy games increases combinatorially with the number of units under the control of the player, making existing search-based methods unable to find good solutions in a reduced time frame. Improving sampling efficiency is one of the main challenges for search-based methods in most domains (and, in particular in strategy games).

Game state and action abstraction [4] are efficient techniques that reduce the search space for search-based methods. Current works on state abstraction [5], [6] and action abstraction [2], [7], [8] have shown their improved performance over non-abstracting algorithms in playing StarCraft. Although these methods gain performance by utilizing abstraction techniques to reduce the search space, domain knowledge is indispensable for them. In the planning domain, methods that require no domain knowledge for abstraction are studied extensively [9]–[15]. However, those have been limited to applications of lesser complexity, e.g. Othello [9]. In our study, we aim to test if similar concepts can be scaled up to strategy games.

In this paper, we propose an algorithm that employs state abstraction by approximate homomorphism [16] for Markov Decision Process (MDP). The generated state abstraction is used to merge tree nodes in Monte Carlo Tree Search (MCTS), which reduces the size of the tree. This paper shows the challenges derived from implementing this approach and the solutions proposed to address them. One of these challenges is that, in order to obtain a good approximate homomorphism, a high number of samples is required. In fact, this number increases with large state and action spaces, which is problematic for strategy games where the state space and action space increases exponentially with the number of units and other elements of the game.

We alleviate this problem in two ways. First, we implement a modification in MCTS (which we call $MCTS_u$) so that, for each node, the algorithm only considers the actions available for one unit (rather than for all of them). Secondly, we introduce an iteration threshold $\alpha_{ABS} \in \mathbb{N}$ that indicates a stopping iteration for the use of abstractions. When the MCTS iteration number $N_{mcts} \in \mathbb{N}$ reaches α_{ABS} , the state abstraction is abandoned and the tree is “expanded” again (abstract nodes are eliminated) to continue the search as in normal MCTS.

Given the fact that the size of the tree changes during search, we call our algorithm Elastic MCTS.

Our contributions can be summarized as follows:

- **Automatic state abstraction for strategy games with no domain knowledge:** Our method applies a state abstraction that requires no domain knowledge for complex environments such as strategy games, in contrast to existing methods which require domain knowledge. While this work focuses on strategy games, the method proposed in this paper may be applicable to other genres, as it does not require game-specific knowledge. We assume the multi-unit setting in this paper while our method can be used for both single-unit and multi-unit settings.
- **An analysis on the effects of state abstraction on the recommendation policy:** Previous works keep the generated abstraction within the tree of MCTS during all iterations. This approach is based on the assumption that the policy resulting from the abstraction is better than the policy from the original state space, neglecting the risk of using a bad-quality state abstraction. Our algorithm sets up an iteration threshold for using the abstractions, which we tune to analyze the impact of turning back to the original tree at different times during the search.

The rest of the paper is structured as follows: In Section II, background knowledge about MCTS, state abstractions and approximate MDP homomorphism is introduced. The Stratega framework is introduced in Section III. Section IV summarizes related works on state abstraction for strategy games and abstractions used in the context of planning. Section V describes Elastic MCTS, and in Section VI we evaluate Elastic MCTS empirically and compare it with other algorithms. Section VII concludes our work and gives ideas for future directions.

II. BACKGROUND

A. Monte Carlo Tree Search (MCTS)

MCTS [3] generates a search tree to estimate the state-action values of the current state. In this tree, each node represents a state and each branch represents an action, with the current state located at the root node. Each node stores the cumulative reward X and the visit count N . Each MCTS iterations consists of 4 phases: selection, expansion, simulation, and backpropagation.

During the selection phase, a tree policy is used to traverse the tree from the root until we reached a node on which we have not yet expanded all possible child nodes. A popular choice for the tree policy is Upper Confidence Bounds (UCB) applied to Trees (UCT) [17]. For a node representing state s , connected with its parent who has an edge representing action a , its UCB value is:

$$UCB1(s, a) = \frac{X(s, a)}{N(s, a)} + C \sqrt{\frac{\ln N_{\text{parent}}}{N(s, a)}}, \quad (1)$$

where the $X(s, a)$ is the cumulative reward, $N(s, a)$ is the visit count of this node, and N_{parent} is the visit count of the parent node. A constant $C \in \mathbb{R}$ controls the trade-off between exploration (selecting nodes of low visit count) and exploitation

(selecting known high-value nodes). The tree policy selects action with the highest UCB1 value, descending the tree until a non-fully-expanded node is reached.

In the expansion phase, a child node is generated that represents the next state retrieved from applying a previously unexplored action. After expansion, it enters the simulation phase, where a roll-out policy will be used to continuously sample actions for a fixed amount of turns or until the end of the game. A classic roll-out policy is the random policy that chooses available actions uniformly. This final state is then evaluated by a state evaluation function and a score R is backpropagated along the path taken during the selection and expansion phases. This value R is normally the game outcome in terminal states, and a value returned by an heuristic function for non-terminal states.

B. State Abstraction and approximate MDP Homomorphism

A Markov Decision Process (MDP) is defined as $\langle \mathbb{S}, \mathbb{A}, R, T, \gamma \rangle$, with state space $\mathbb{S}$, action space $\mathbb{A}$, reward function $R : \mathbb{S} \times \mathbb{A} \mapsto \mathbb{R}$, transition function $T : \mathbb{S} \times \mathbb{A} \times \mathbb{S} \mapsto \mathbb{R}$, and discount factor $\gamma \in \mathbb{R}, 0 < \gamma < 1$ for discounting future rewards. State Abstraction for MDPs can be formalized as a mapping $\phi(s) = s_\phi, s \in \mathbb{S}, s_\phi \in \mathbb{S}_\phi$. The $\mathbb{S}_\phi$ is an abstract state space. Usually, we wish the size $|\mathbb{S}_\phi| < |\mathbb{S}|$ to reach a better sample efficiency or a shorter searching time. The approximate MDP homomorphism [16] defines the similarity between two states s_1, s_2 by defining two approximation errors:

$$\epsilon_R(s_1, s_2) = \max_a |R(s_1, a) - R(s_2, a)| \leq \eta_R \quad (2)$$

$$\epsilon_T(s_1, s_2) = \sum_{s'} |T(s'|s_1, a) - T(s'|s_2, a)| \leq \eta_T \quad (3)$$

ϵ_R is the reward approximation error, while ϵ_T is the transition approximation error. η_R and η_T are the respective approximation thresholds. Two states are considered *similar* if they hold that $\epsilon_R(s_1, s_2) \leq \eta_R$ and $\epsilon_T(s_1, s_2) \leq \eta_T$. In MCTS, *similar* states from the same depth are considered candidates to construct a local approximate homomorphism. For each depth of the tree, two or more similar original (or *ground*) states can be grouped into the same *abstract state* or node. The reward and visiting count of the abstract node are $\hat{X} = \frac{\sum X_i}{m}$ and $\hat{N} = \frac{\sum N_i}{m}$, where m is number of original nodes in this abstract node. When a new ground node is added to an abstract node, the statistics of this abstract node will be updated accordingly.

III. STRATEGA

The Stratega [18] framework (see screenshot in Figure 1) is developed for studying AI agents in general strategy game playing. Stratega uses an isometric view for the battlefield, where there are different tiles for game elements: landforms, buildings, resources, and army units. It allows developers to create their own turn-based and real-time strategy games through the YAML markup language, by setting up game elements and their parameters. One of the important features of Stratega is that it provides a forward model that can be used by statistical forward planning methods, such as MCTS or Rolling Horizon Evolution [19].



Fig. 1. A classic scene of Stratega framework.

We use the game *Kill The King* from Stratega for the evaluation of the search methods presented in this paper. *Kill The King* is a turn-based, two-player strategy game where each player commands their units to defeat the opponent's king for a win. We limited the maximum number of turns to 100. If after 100 turns and no king dies, the game ends and returns a draw. In this game, we designed 4 different unit types: *King*, *Warrior*, *Archer*, *Healer*. *King*, *Warrior* and *Archer* share the same action types: [*Move*, *Attack*, *Do-nothing*]. The action types for *Healer* are: [*Move*, *Heal*, *Do-nothing*]. The action space of a unit depends on their action types and unit attributes. For example, the *Move Range* of *King* is set to 2, resulting in 12 surrounding tiles for its *Move* action. Its *Attack Range* is 2, enabling it to choose any opponent unit to attack within 2 tiles. In this case, the maximum size of its action space is $(12 + 1) \times (12 + 1) = 169$, as units can first move (+1 for not moving) and then attack (+1 for not attacking). A player, at the beginning of their turn, can act with n units, providing a combinatorial space bounded at 169^n for a turn. In this paper, we experiment with a number of units between 4 and 11, which constitutes an action space bounded between 10^5 and 10^{14} actions. The game is also complex from a strategic point of view: different unit types have distinct attribute values, which makes their behavior different and forces units to use different strategies. The attribute set for this game consists of: *Health Points*, *Move Range*, *Attack Range*, *Attack Damage*, *Heal Strength*. The maps for Stratega are grid-like 1, where tiles such as mountains or water block the way, requiring an aspect of coordination between units for effective movement.

Kill The King is chosen to evaluate our proposed algorithm because of the following characteristics. Firstly, although it is not as complex as battles in Starcraft, these two games share challenges such as combinatorial state and action space, which is common in most strategy games. Second, the search space can be controlled by the number of units. Therefore, by increasing the number of units, we can test how the performance of our method scales with the increasing complexity of the search space and the size of the tree. Furthermore, varieties of the game can be created by changing the unit composition. These varieties are used to evaluate the methods by searching strategies for different environments.

IV. RELATED WORK

State abstraction is a popular technique that shows its application in strategy games in the following works. [20] proposed a Monte Carlo planning algorithm to play *Capture The Flag* game. To reduce the planning complexity, a handcrafted game state abstraction that divides the game map into tiles is used. [21] proposed a method that presents the map of StarCraft with regions connected by checkpoints, largely simplifying the state space. [5] combines the state abstraction in [21] and action abstraction in playing Starcraft combats. Combat-irrelevant units such as workers and buildings are removed from high-level game state representation. With the search space reduced by their abstraction, their search-based method shows a performance close to a script-based agent. [22] proposes to encode the game state with vectors that contain information about entities. This representation enables eliminating superfluous information and grouping nodes with the same vector.

While the works mentioned above show that state abstraction is a powerful tool in complex searching space, [6] investigates the effect of different state abstractions. In their work, 4 different state abstractions are created and they show different performance with MCTS in StarCraft. Among these works, different kinds of state abstraction methods are proposed. However, most of them require domain knowledge to construct the state abstraction. [22] utilizes a parameter optimizer to pick up entity information used for state abstraction, avoiding the use of human knowledge. However, this method shows no clear performance improvement in their evaluation.

Most existing state abstraction applications in strategy games depend on domain knowledge. In the planning domain, state abstraction that requires no domain knowledge is more common. [9] proposed approximate MDP homomorphism to construct state abstraction for MCTS. According to the state abstraction, states in each depth of MCTS that have a similar transition function and reward function are grouped. The similarity definition for this approach is shown in Eqs. 2 and 3. The approximate homomorphism is generated from the samples collected by MCTS in a batch manner. Although their approach is shown to improve the performance in the planning domain, it requires extra action abstraction and pruning when applied to a board game such as Othello. [11] applies the approximate homomorphism to state-action abstraction, where the state-action pairs are grouped in MCTS.

Progressive Abstraction Refinement for Sparse Sampling (PARSS) [12], [14] is a method that starts with a coarse state abstraction where all the states are clustered in the same group and refining it progressively. [13] proposed *On-The-Go* abstraction that abandons the batch style of updating the abstraction [9], [11]. In their approach, the maintained abstraction is updated more frequently. In their work, a variable *recent counts* for counting the number of visits is stored in each tree node. When it reaches a pre-defined threshold α_{otg} , the abstraction for this state is updated and its *recent counts* is reset. [15] proposed Abstraction Refining that rejects adding a

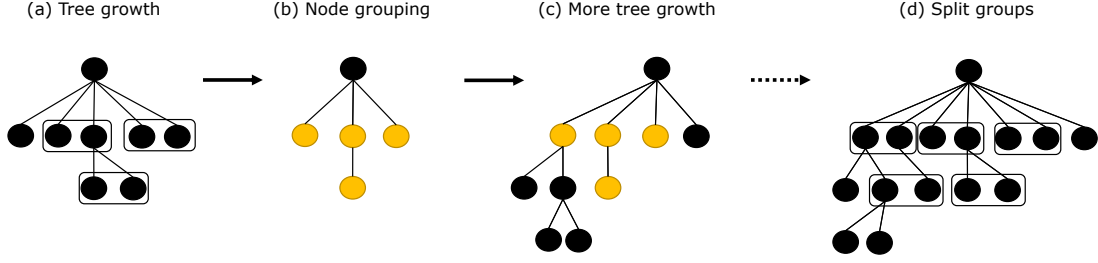


Fig. 2. Overview of dynamic changes of tree nodes in Elastic MCTS. Ground nodes are *black* while abstract nodes are *yellow*. At first, the tree grows as in normal MCTS (a). After a number of B iterations, ground nodes are grouped by using Approximate MDP Homomorphism (b). Search then continues adding more ground node (c), repeating state aggregation after every B iterations. When the abstraction iteration threshold is reached, abstract nodes are split (d) and the search continues to explore the original game without using the state abstraction until the thinking budget expires.

state that is similar to a known state in the tree. Their approach improves the performance of MCTS in stochastic environments. In conclusion, these methods are general and can be applied to different tasks. However, they are designed for planning domains and did not show if they can be applied to complex search spaces such as those from strategy games.

V. ELASTIC MCTS

We propose *Elastic MCTS*, which combines approximate MDP homomorphism with MCTS, for strategy games. Our method is described in detail in Section V-C and depicted in Figure 2. Before this, we focus on two modifications that are needed to adapt the principles of MDP homomorphism to the large combinatorial search spaces present in this domain: unit ordering to reduce the action space (Section V-A) and introducing an iteration threshold to revert to the original (ground) state space.

A. Approximate Homomorphism in Strategy Games

Our method is based on state abstraction in the planning domain [9], where a local approximate homomorphism (see II-B for details) is constructed and constantly updated from trajectories sampled by MCTS. However, two issues occur when this method is applied to complex search spaces such as those from strategy games. The first issue is that the number of samples required to generate a good-quality approximate homomorphism depends on the sizes of the state and action spaces. In strategy games, where the state space is combinatorial and the action space changes according to different game states, the small number of samples collected within a limited decision budget results in a bad-quality state abstraction.

The second issue is that the reward and transition approximation errors ϵ_R and ϵ_T (Eqs. 2 and 3) are calculated executing all possible actions available in two states. For two states that have different action spaces, the original definitions of ϵ_R and ϵ_T conflict with the fact that most actions are likely not legal in *both* states s_1 and s_2 at the same time. It is quite common in strategy games to observe different states with different sets of available actions. While it would be possible to resolve the approximation errors by only using the (small) set of common actions, the resulting values would not represent the true futures of both states in terms of rewards and transitions. Our initial

tests (not included in this paper) showed that this indeed does not provide good abstractions for MCTS.

We alleviate these two issues by implementing a variant of MCTS called $MCTS_u$. In $MCTS_u$, each node corresponds to a state where a single unit can move. The node’s edges are actions available to said unit only. Consequently, the action space for states candidate for merging is much smaller than the original combinatorial action space. Moreover, candidate states have a large proportion of common actions in their action space because they represent states where the same units act.

A consideration for $MCTS_u$ is to decide the units’ move order. In our implementation, the move order (by the sole purpose of the agent’s search process) is set randomly at the beginning of the game and kept fixed during the whole game. While, theoretically, this ordering removes the guarantee of optimal convergence, empirically the advantage obtained by reducing the action space creates stronger players. This is shown in the results discussed in Section VI and is in line with previous results in the literature [23].

B. Elastic State Grouping and Un-grouping

We apply the constructed state abstraction to MCTS for grouping tree nodes. As [9], the state abstraction in our method is also constructed in a batch manner: for every B iterations of MCTS, the sampled trajectories are used to construct an approximate homomorphism, which aggregates similar nodes into groups according to the states they represent. Each MCTS node stores statistics including cumulative reward and visit count. For a node group, it stores these as the average of the statistics of all its ground nodes. The following MCTS iterations will be guided by these group statistics in two ways. First, the UCT value of one node for the selection is calculated based on the statistics of the group it belongs to. Second, when updating a node’s statistic during back-propagation, their group nodes also update their statistics.

Existing methods keep using the generated state abstraction until the search is finished. However, abstractions are known to introduce imperfections in the search space [13]. With an erroneous abstraction, the policy obtained from abstraction performs worse than the policy derived from the ground search space. $MCTS_u$ reduces the size of action space and helps

Algorithm 1 Elastic MCTS. $N_{fm} \in \mathbb{N}$: maximum forward model calls. $\alpha_{ABS} \in \mathbb{N}$: MCTS iteration threshold. $N_{mcts} \in \mathbb{N}$: current MCTS iteration number. $B \in \mathbb{N}$: batch size. $\phi(s) = \hat{s}$: state abstraction that maps an MCTS node representing s to a node group $\hat{s}$. $\eta_R \in \mathbb{R}$ and $\eta_T \in \mathbb{R}$ are the reward function error threshold and transition error threshold, respectively.

Require: $N_{fm}, \alpha_{ABS}, \eta_R, \eta_T$

```

1:  $\phi := s \rightarrow \hat{s}, \hat{s} = \{s\}$  # Initialize the abstraction
2: while  $USED\_FMCALL < N_{fm}$  do
3:    $MCTSIteration(\phi)$ 
4:   if  $N_{mcts} > \alpha_{ABS}$  then
5:      $\phi := s \rightarrow \hat{s}, \hat{s} = \{s\}$  # Fig.2 (d)
6:   else if  $N_{mcts} \% B == 0$  then
7:      $\phi = ConstructAbstraction(\phi, \eta_R, \eta_T)$  # Fig.2 (b)
8:    $N_{mcts} = N_{mcts} + 1$ 

```

construct a better abstraction, but these imperfections remain. Another issue is that states in the same abstract node share their statistics while grouped, forcing the action selection for the recommendation policy of MCTS to choose effectively at random among actions that lead to the same group node.

We propose a novel approach to solve both problems mentioned above. We set up an iteration threshold for abstraction α_{ABS} . After α_{ABS} iterations, MCTS assigns the node group statistics to each node in this group and abandons the state abstraction (breaking the node groups into ground tree nodes). The remaining MCTS iterations follow the normal MCTS algorithm. At this point, MCTS does not search in an imperfect space and nodes split from the same group might be now independently visited. The action-state values can become different and MCTS’s recommendation policy can distinguish better among the available actions to suggest.

C. Elastic MCTS

Algorithm 1 shows the pseudocode of Elastic MCTS, which runs MCTS iterations (line 3) until the budget is exhausted (given in forward model calls, line 2). The set of abstract nodes is initialized to set of the ground states (line 1), and is updated after every batch of B iterations (line 7). When the number of iterations surpasses α_{ABS} (line 4), we abandon state abstraction to return to the original ground search space (line 5), by splitting all the nodes from state groups and assigning statistics of the state group to the ground nodes. This procedure is also depicted in Figure 2.

Algorithm 2 shows the MCTS iteration step. This only differs from normal MCTS in that it uses the statistics (cumulative reward and visit count) of the node group rather than the original tree nodes, for the selection (line 1) and backpropagation (line 9) steps. Note that new nodes added in the expansion phase are added as ground nodes to the tree, merging into states after B iterations as shown in Algorithm 1.

Algorithm 3 updates the state abstraction: from leaf nodes to the root (line 1), for every ground tree node that is not part of an abstract node (line 2), a similarity check is performed against all sibling abstract nodes. This similarity is determined

Algorithm 2 MCTSIteration(ϕ)

```

1: while Select a child  $K$  with maximum UCT
   value with  $\phi: V_{uct} = X(\phi(s), a)/N(\phi(s), a) +$ 
    $C\sqrt{\ln N_{parent}/N(\phi(s), a)}$ . do
2:   if Node  $K$  is not fully-expanded then
3:     Expand this node by generating a new child node  $P$ .
4:     Rollout for  $P$  and obtain reward  $R$  from state
     evaluation function.
5:     Break the while loop.
6:   else if Node  $K$  represents the end of game then
7:     Obtain reward  $R$  from state evaluation function.
8:     Break the while loop.
9: Backpropagate  $R$ , updating  $X(s, a)$  and  $N(s, a)$  for node
   group  $\phi(s)$  in selection path.

```

Algorithm 3 ConstructAbstraction(ϕ, η_R, η_T), l is the tree depth and L is the maximum depth of the current tree.

```

1: for  $l = L$  to 1 do
2:   for all state  $s_1$  in depth  $l$  that is not grouped do
3:     for all abstract state  $\hat{s}$  in  $\phi$  do
4:        $s_{1\_in\_}\hat{s} = \text{true}$ 
5:       for all state  $s_2$  in  $\hat{s}$  do
6:          $\epsilon_R = \max_a |R(s_1, a) - R(s_2, a)|$ 
7:          $\epsilon_T = \sum_{s'} |T(s'|s_1, a) - T(s'|s_2, a)|$ 
8:         if  $\epsilon_R > \eta_R$  or  $\epsilon_T > \eta_T$  then
9:            $s_{1\_in\_}\hat{s} = \text{false}$ , break
10:        if  $s_{1\_in\_}\hat{s} == \text{true}$  then
11:          Add  $s_1$  in abstraction node
12:        else
13:          Create a new abstract node

```

by computing the values of two errors ϵ_R and ϵ_T from samples. The samples from MCTS are triplets that consist of a state, an action and a return: $\langle s, a, R \rangle$. For computing the ϵ_R error between two states s_1 and s_2 , we calculate $|R(s_1, a) - R(s_2, a)|$ for each action a present in either action space of s_1 and s_2 , with $R(s_i, a) = 0$ if a is invalid for any s_i . ϵ_R takes the value of the maximum difference found (line 5). ϵ_T is also calculated for all actions (line 6), with $|T(s'|s_1, a) - T(s'|s_2, a)| = 0$ only when the s_1 and s_2 both have action a available and this action leads to the same next state s' . Because we only consider two state representing the same unit for state abstraction, their next state s'_1 and s'_2 are recognized the same when the unit-related attributes have the same values.

If an abstract node is found where $\epsilon_R \leq \eta_R$ and $\epsilon_T \leq \eta_T$, we add the ground node to this node (lines 7 – 8). If no group fulfills this condition, we create a new group including only the ground node in it (line 10).

VI. EXPERIMENTS

Four agents are used for the experiments in this paper:

- 1) *Combat Agent*: This agent is based on a rule-based agent [18] built-in Stratega. Its strategy is to concentrate attacks to a single isolated enemy unit and assign healers to heal the strongest ally units. The isolation score for a

unit depends on the number of nearby ally and enemy units. Once a target is chosen, the agent searches available action for each agent to i) get close to the target, ii-a) attack the target or ii-b) heal the target.

- 2) *MCTS*: the default MCTS algorithm with no abstraction nor unit ordering.
- 3) *MCTS_u*: MCTS with unit ordering, no state abstraction.
- 4) *Elastic MCTS_u*: Elastic MCTS with fixed unit ordering.

The same *state evaluation function* is used by *MCTS*, *MCTS_u* and *Elastic MCTS_u*. This function gives an utility value $0.0 \leq R \leq 1.0$ to a state after normalization. The utility values for win, loss and draw are 1, -1, 0, respectively. If the given state is not terminal, the value of the state is $R = 1 - \frac{dh}{DH}$, where d is the distance between player's units and opponent's king, h the health point of opponent's king, and D and H are the maximum values of d and h . In conclusion, our state evaluation function encourages the agent to get close to opponent's king and attack it.

A. Parameter Optimization for Agents with NTBEA

All our agents are pitched against each other in *1vs1* games. We use the optimiser N-Tuple Bandit Evolutionary Algorithm (NTBEA) [24] to tune the parameters of each agent. NTBEA utilizes a combinatorial multi-armed bandit to navigate the parameter space, while building a landscape model for a noisy evaluation function.

For NTBEA, we set the exploration factor for the multi-armed bandit to 2, we use 50 neighbors, and a limit of 50 iterations. When tuning the agent parameters, the fitness is set as the win rate of the agent playing against *CombatAgent*. In these and all the following experiments, the computational budget for each action decision by all search agents is set to 30,000 forward model calls. As an opponent model, all search agents simply pick valid actions uniformly at random. Additionally, for *Elastic MCTS_u*, we empirically determined the error approximation thresholds for $\eta_R = 0.1$ and $\eta_T = 0.3$.

The following describes the parameter space explored by NTBEA: *MCTS* and *MCTS_u* have the same parameter spaces: we evaluate the exploration factor $C \in \{0.1, 1, 10, 100\}$ and rollout length $L \in \{20, 40, 60, 80, 100\}$. *Elastic MCTS_u* adds to these two parameters (C and L) the batch size $B \in \{20, 40, 60\}$ and the iteration threshold to stop using abstractions $\alpha_{ABS} \in \{4 \times B, 8 \times B, 12 \times B, 16 \times B\}$. The parameters found are $\{C = 0.1, L = 20\}$, $\{C = 10, L = 100\}$ and $\{C = 0.1, L = 40, B = 20, \alpha_{ABS} = 12\}$ for *MCTS*, *MCTS_u* and *Elastic MCTS_u*.

B. Algorithmic Performance

Using the parameter values as tuned by NTBEA, agents are evaluated by playing against each other in different variants of *KillTheKing*. We designed two groups of experiments to evaluate the performance of the proposed method. The first group evaluates the presented agents across scenarios with different amount of units, to observe how this performance changes as the action space becomes larger. The second group investigates the performance in maps with different layouts.

TABLE I
WIN RATES WITH STANDARD DEVIATION FOR GAMES WITH 1 KING, 1 WARRIOR, 1 ARCHER AND 1 HEALER.

Agent 1	Agent 2	Agent 1	Agent 2
MCTS	Combat Agent	$54.6 \pm 4.7\%$	$45.4 \pm 4.7\%$
MCTS _u	Combat Agent	$67.2 \pm 2.9\%$	$32.6 \pm 2.9\%$
Elastic MCTS _u	Combat Agent	$71.3 \pm 3.1\%$	$28.7 \pm 3.1\%$
MCTS _u	MCTS	$66.0 \pm 5.7\%$	$25.8 \pm 5.0\%$
Elastic MCTS _u	MCTS	$65.8 \pm 4.0\%$	$30.4 \pm 4.4\%$
Elastic MCTS _u	MCTS _u	$51.6 \pm 6.4\%$	$40.8 \pm 8.6\%$

TABLE II
WIN RATES WITH STANDARD DEVIATION FOR GAMES WITH 1 KING, 2 WARRIORS, 2 ARCHERS AND 2 HEALERS

Agent 1	Agent 2	Agent 1	Agent 2
MCTS	Combat Agent	$55.8 \pm 3.2\%$	$44.2 \pm 3.2\%$
MCTS _u	Combat Agent	$74.4 \pm 2.4\%$	$25.6 \pm 2.4\%$
Elastic MCTS _u	Combat Agent	$77.4 \pm 2.1\%$	$22.4 \pm 1.7\%$
MCTS _u	MCTS	$72.0 \pm 2.8\%$	$21.8 \pm 2.4\%$
Elastic MCTS _u	MCTS	$69.0 \pm 4.5\%$	$27.0 \pm 3.0\%$
Elastic MCTS _u	MCTS _u	$54.4 \pm 3.2\%$	$37.0 \pm 3.4\%$

Win Rates for both players are reported (draw rates can be inferred). Each win rate is shown with its standard deviation, obtained by running each pairing 5 times with 5 different seeds on the same setting (game map, starting unit positions, etc.).

In the experiment for different unit numbers, we have 3 army compositions: (*1 King, 1 Warrior, 1 Archer, 1 Healer*), (*1 King, 2 Warriors, 2 Archers, 2 Healers*) and (*1 King, 3 Warriors, 3 Archers, 3 Healers*). For each army composition, 50 game levels based on the map "lak110d" [25] are generated by randomly choosing initial positions for each unit. In total, there are 500 games played for each army composition (5 seeds, 50 game levels, $\times 2$ from switching starting sides).

Tables I, II and III show the experimental results. As can be seen in Table I, all three *MCTS*, *MCTS_u* and *Elastic MCTS_u* clearly outperform the *Combat Agent*. When playing against *MCTS*, *MCTS_u* and *Elastic MCTS_u* both outperform *MCTS* by a large margin. The win rate of *MCTS_u* is slightly higher than the win rate of *Elastic MCTS_u*, but the difference is at a scale of the standard deviation. In the results of games played between *Elastic MCTS_u* and *MCTS_u*, we can see a higher winning rate for *Elastic MCTS_u*, with a difference larger than 10.0%. This consistently shows the improvement obtained by introducing state abstraction to the algorithm, with *Elastic MCTS_u* showing good performance against all other agents.

When the search complexity increases from 4 units to 7 units (Table II) and 10 units (Table III), the results mentioned above remain consistent. It is worth noting that the *Elastic MCTS_u* performs better when the number of units goes up (see results when playing against *Combat Agent* and *MCTS_u*), which indicates that our method for state abstraction is able to scale appropriately when using more units.

To evaluate the performance of the proposed method in different maps, we choose 27 maps from [25]. 5 army compositions from 4 to 11 units are tested in each map (see Table IV).

TABLE III

WIN RATES WITH STANDARD DEVIATION FOR GAMES WITH 1 KING, 3 WARRIORS, 3 ARCHERS AND 3 HEALERS

Agent 1	Agent 2	Agent 1	Agent 2
MCTS	Combat Agent	59.6 $\pm$ 2.6%	40.4 $\pm$ 2.6%
MCTS _u	Combat Agent	84.8 $\pm$ 2.5%	15.2 $\pm$ 2.5%
Elastic MCTS _u	Combat Agent	82.6 $\pm$ 2.3%	17.4 $\pm$ 2.3%
MCTS _u	MCTS	77.6 $\pm$ 3.1%	15.2 $\pm$ 3.1%
Elastic MCTS _u	MCTS	75.8 $\pm$ 2.5%	20.8 $\pm$ 2.3%
Elastic MCTS _u	MCTS _u	49.1 $\pm$ 1.9%	38.8 $\pm$ 2.7%

TABLE IV

WIN RATES WITH STANDARD DEVIATION FOR GAMES WHERE ELASTIC MCTS_u PLAYED AGAINST MCTS_u IN 27 DIFFERENT LAYOUTS. THE ARMY COMPOSITION COLUMN INDICATES THE NUMBER OF WARRIORS (XW), ARCHERS (XA), HEALERS (XH) AND 1 KING (K).

Army Composition	Elastic MCTS _u	MCTS _u
K3H	61.1 $\pm$ 8.3%	33.3 $\pm$ 7.4%
K3W3A3H	51.1 $\pm$ 6.8%	30.0 $\pm$ 4.1%
K10A	59.3 $\pm$ 5.5%	25.2 $\pm$ 3.8%
K10W	54.8 $\pm$ 4.9%	35.2 $\pm$ 6.1%
K5W5A	53.0 $\pm$ 4.5%	30.7 $\pm$ 5.7%

We evaluate the performance of Elastic MCTS_u playing against MCTS_u. Table IV shows the average win rate with its corresponding standard deviation for both agents in each army composition. In all games, Elastic MCTS_u outperforms MCTS_u by large margins (between 19.6% and 34.1%). These results show that the proposed method improves the performance of the algorithm and its improvements are consistent in different army compositions and game maps.

C. Influence of Abstraction Threshold

To investigate the influence of different iteration thresholds, we pitch Elastic MCTS_u against MCTS_u, with the map settings and army compositions used for the experiments whose results are shown in Tables I, II and III. The values used for the agents' parameters are those obtained by NTBEA, as mentioned in Section VI-A, with the exception of the iteration threshold α_{ABS} . The values of this parameter are explored in this experiment to observe the effect of using state abstraction during different proportions of MCTS iterations. We try with proportions $\alpha_{ABS} \in \{0\%, 25\%, 50\%, 75\%, 100\%\}$. When it is set to 0%, the algorithm behaves like (normal) MCTS. When it is 100%, the state abstraction is used during all MCTS iterations and the recommendation policy picks an action based on statistics from the group nodes.

As Table V shows, the performance of the Elastic MCTS_u agent differs when the abstraction threshold changes. Note these two agents use different hyper-parameters (tuned by NTBEA) thus their win rates differ with proportion set to 0%. There seems to be a sweet spot on the value of this threshold: performance is best when using the state abstraction during the first 50% and 75% iterations of Elastic MCTS_u, obtaining lower winning rates when closer to 0% and 100%. This result is especially noticeable for a threshold value of 100%, where

TABLE V

WIN RATES WITH STANDARD DEVIATION OF ELASTIC MCTS_u VS MCTS_u. THE PROPORTION COLUMN INDICATES AT WHICH % OF THE SEARCH STATE ABSTRACTION IS ABANDONED.

Army Composition	Proportion	Elastic MCTS _u	MCTS _u
KWAH	0%	47.4 $\pm$ 2.4%	46.0 $\pm$ 3.8%
	25%	50.0 $\pm$ 2.7%	40.4 $\pm$ 3.1%
	50%	51.6 $\pm$ 4.6%	39.4 $\pm$ 3.8%
	75%	51.8 $\pm$ 4.4%	38.2 $\pm$ 6.0%
	100%	39.0 $\pm$ 2.8%	45.0 $\pm$ 1.8%
K2W2A2H	0%	49.0 $\pm$ 3.6%	42.6 $\pm$ 4.0%
	25%	49.8 $\pm$ 3.9%	42.6 $\pm$ 4.8%
	50%	52.8 $\pm$ 4.8%	37.6 $\pm$ 4.1%
	75%	52.8 $\pm$ 3.6%	39.2 $\pm$ 6.4%
	100%	38.8 $\pm$ 3.3%	47.2 $\pm$ 1.5%
K3W3A3H	0%	49.0 $\pm$ 3.8%	38.8 $\pm$ 3.5%
	25%	47.8 $\pm$ 4.3%	39.0 $\pm$ 3.8%
	50%	55.3 $\pm$ 3.9%	33.8 $\pm$ 3.3%
	75%	49.2 $\pm$ 3.3%	40.2 $\pm$ 3.1%
	100%	40.0 $\pm$ 2.3%	44.4 $\pm$ 1.4%

the win rate is the lowest one for all army compositions. In all these, MCTS_u obtains a consistently higher winning rate against Elastic MCTS_u. Our interpretation of this phenomenon is that nodes sharing the same statistics in abstracted nodes until the end of the decision process makes the recommendation policy of Elastic MCTS_u behave suboptimally, as there is not enough information to discern between the different available actions at the root node. However, abandoning the abstraction at an intermediate point during the search lands better results than MCTS_u also in all army compositions, showing the usefulness of Elastic MCTS and these abstractions.

D. Compression Rate

To show the difference between the ground and the abstracted state space, we define a compression rate as N_{tree}/N_{abs_tree} , where N_{tree} is the number of nodes generated by MCTS and N_{abs_tree} is the number of node groups generated by the abstraction. Note that with the same tree from MCTS, fewer groups mean more nodes are grouped together and the size of the tree is reduced more. We evaluate this compression rate in 20 instances of the map "lak110d" with the army composition (1 King, 1 Warrior, 1 Archer, 1 Healer). Figure 3 visualizes the achieved compression rate against MCTS iterations, showing a moderate increase of compression rate over time. In our previous experiments, the state abstraction was abandoned in the 240th iteration ($\alpha_{ABS} \times B = 12 \times 20$), corresponding to a compression rate of 10 states per group node.

The state abstraction does not influence the computing time significantly. The average decision times are 667 ± 13 and 685 ± 13 ms for MCTS_u and Elastic MCTS_u, respectively.

VII. CONCLUSION AND FUTURE WORK

In this paper, we propose a variant of MCTS that generates clusters of nodes by using state abstraction. Our work is inspired by methods that use state abstraction in planning domains, but it is adapted to target complex search and action spaces, while still requiring no domain knowledge. Our experiments show

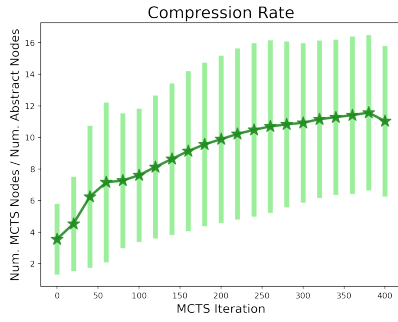


Fig. 3. Compression Rates with standard deviation from 40 game plays.

that the proposed method outperforms MCTS and a baseline rule-based agent in multiple variants of a turn-based strategy game, *Kill the King*. We also observe a reduction of the tree size by a factor of 10 and a considerable improvement in the agent’s performance when using the state abstraction during a proportion of the search $< 100\%$.

The results shown in this paper have been obtained in a turn-based game, but we hypothesize that the gains in performance may also benefit other real-time or more complex turn-based strategy games. Due to the increased number of units controlled in games like Starcraft and the restricted time to return an action, we believe that the observed efficiency improvements may transfer well to RTS games. This is an immediate case for future work, where we’ll investigate the applicability and scalability of Elastic MCTS to more complex games.

Besides aiming to improve the performance of the game playing agent, other aspects can also be investigated. For example, it is interesting to observe if different gameplay styles can be obtained when using these abstractions or if, on the contrary, the simplifications made in the tree prevent us from that goal. Recent works on quality-diversity methods applied to strategy games could be explored in conjunction with Elastic MCTS [26]. Additionally, one of the most widespread methods for strategy games and large action spaces is the use of portfolio methods [2], and it will be interesting to see the potential synergies of state abstraction with these algorithms. Finally, it is also possible to try different mechanisms for state abstraction (e.g. [12]), which could be an alternative to Approximate MDP Homomorphism. In fact, considering the complexity of strategy games, it’s possible that different abstractions can be applicable to different moments of the game, which lies an interesting line of research ahead to discover which abstraction methods can be applied to which game situations.

ACKNOWLEDGMENTS

Work supported by UK EPSRC grant EP/T008962/1.

REFERENCES

- [1] David Churchill and Michael Buro. Build order optimization in starcraft. In *Seventh Artificial Intelligence and Interactive Digital Entertainment Conference*, 2011.
- [2] David Churchill and Michael Buro. Portfolio greedy search and simulation for large-scale combat in starcraft. In *2013 IEEE Conference on Computational Intelligence in Games (CIG)*, pages 1–8. IEEE, 2013.
- [3] Guillaume Chaslot, Sander Bakkes, Istvan Szita, and Pieter Spronck. Monte-carlo tree search: A new framework for game ai. In *Proceedings of the AAAI Conference on Artificial Intelligence and Interactive Digital Entertainment*, volume 4, pages 216–217, 2008.
- [4] George Konidaris. On the necessity of abstraction. *Current opinion in behavioral sciences*, 29:1–7, 2019.
- [5] Alberto Uriarte and Santiago Ontanón. Game-tree search over high-level game states in rts games. In *Tenth Artificial Intelligence and Interactive Digital Entertainment Conference*, 2014.
- [6] Alberto Uriarte and Santiago Ontanón. High-level representations for game-tree search in rts games. In *Tenth Artificial Intelligence and Interactive Digital Entertainment Conference*, 2014.
- [7] Nicolas Arturo Barriga, Marius Stanescu, and Michael Buro. Puppet search: Enhancing scripted behavior by look-ahead search with applications to real-time strategy games. In *Eleventh Artificial Intelligence and Interactive Digital Entertainment Conference*, 2015.
- [8] Nicolas A Barriga, Marius Stanescu, and Michael Buro. Combining strategic learning with tactical search in real-time strategy games. In *Thirteenth Artificial Intelligence and Interactive Digital Entertainment Conference*, 2017.
- [9] Nan Jiang, Satinder Singh, and Richard Lewis. Improving uct planning via approximate homomorphisms. In *Proceedings of the 2014 international conference on Autonomous agents and multi-agent systems*, pages 1289–1296, 2014.
- [10] Jesse Hostetler, Alan Fern, and Tom Dietterich. State aggregation in monte carlo tree search. In *Twenty-Eighth AAAI Conference on Artificial Intelligence*, 2014.
- [11] Ankit Anand, Aditya Grover, Parag Singla, et al. Asap-uct: Abstraction of state-action pairs in uct. In *Twenty-Fourth International Joint Conference on Artificial Intelligence*, 2015.
- [12] Jesse Hostetler, Alan Fern, and Thomas G Dietterich. Progressive abstraction refinement for sparse sampling. In *UAI*, pages 365–374, 2015.
- [13] Ankit Anand, Ritesh Noothigattu, Parag Singla, et al. Oga-uct: On-the-go abstractions in uct. In *Proceedings of the International Conference on Automated Planning and Scheduling*, volume 26, 2016.
- [14] Jesse Hostetler, Alan Fern, and Thomas Dietterich. Sample-based tree search with fixed and adaptive state abstractions. *Journal of Artificial Intelligence Research*, 60:717–777, 2017.
- [15] Samuel Sokota, Caleb Y Ho, Zaheen Ahmad, and J Zico Kolter. Monte carlo tree search with iteratively refining state abstractions. *Advances in Neural Information Processing Systems*, 34:18698–18709, 2021.
- [16] Balaraman Ravindran and Andrew G Barto. Approximate homomorphisms: A framework for non-exact minimization in markov decision processes. 2004.
- [17] Levente Kocsis and Csaba Szepesvári. Bandit based monte-carlo planning. In *European Conf. on machine learning*, pages 282–293. Springer, 2006.
- [18] Alexander Dockhorn, Jorge Hurtado Ghueso, Dominik Jeurissen, and Diego Perez Liebana. Stratega: A general strategy games framework. In *AIIDE Workshops*, 2020.
- [19] Alexander Dockhorn, Diego Perez-Liebana, et al. Portfolio search and optimization for general strategy game-playing. In *2021 IEEE Congress on Evolutionary Computation (CEC)*, pages 2085–2092. IEEE, 2021.
- [20] Michael Chung, Michael Buro, and Jonathan Schaeffer. Monte carlo planning in rts games. In *CIG*. Citeseer, 2005.
- [21] Gabriel Synnaeve and Pierre Bessiere. A bayesian tactician. In *Computer Games Workshop at ECAI 2012*, pages pp–114, 2012.
- [22] Alexander Dockhorn, Diego Perez-Liebana, et al. Game state and action abstracting monte carlo tree search for general strategy game-playing. In *2021 IEEE Conference on Games (CoG)*, pages 1–8. IEEE, 2021.
- [23] Naoyuki Sato and Kokoro Ikeda. Three types of forward pruning techniques to apply the alpha beta algorithm to turn-based strategy games. In *2016 IEEE Conference on Computational Intelligence and Games (CIG)*, pages 1–8, 2016.
- [24] Simon M Lucas, Jialin Liu, and Diego Perez-Liebana. The n-tuple bandit evolutionary algorithm for game agent optimisation. In *2018 IEEE Congress on Evolutionary Computation (CEC)*, pages 1–9, 2018.
- [25] Nathan R Sturtevant. Benchmarks for grid-based pathfinding. *IEEE Trans. on Computational Intelligence and AI in Games*, 4(2):144–148, 2012.
- [26] Diego Perez-Liebana, Cristina Guerrero-Romero, et al. Generating diverse and competitive play-styles for strategy games. In *2021 IEEE Conference on Games (CoG)*, pages 1–8. IEEE, 2021.